NYCU-EE ICLAB – FALL 2025

Lab09 Exercise: Design and Verification Using SystemVerilog

Design: Role-Playing Game

Data Preparation

1. Extract the test data from TA's directory:

   % tar xvf ~iclabTA01/Lab09.tar

2. The extracted LAB directory contains:

   Exercise/

Design Description

This assignment aims to simulate the management of player status in a role-playing game. Each player's information is stored in DRAM, including five attributes and login date. Different operations will update the data based on predefined rules, ensuring that values remain within valid limits.

The system supports several actions. In the **Login** action, players update their login date, and continuous login rewards them with Experience (Exp) and Mana Point (MP). In the **Level Up** action, different modes and training types determine the formulas used to increase attributes. In the **Battle** action, damage to both player and monster is computed, updating Hit Points (HP) and rewarding or penalizing Exp and MP depending on success or defeat. The **Use Skill** action consumes MP according to a skill list. Lastly, the **Check Inactive** action monitors account activity.

Operations:

- **Login**

  Input:

  1. Today's date.
  2. Select the No. of player in DRAM.

- **Level up**

  Input:

  1. Choose the type of training.
  2. Select the scene mode.
  3. Select the No. of player in DRAM.

- **Battle**

  Input:

  1. Select the No. of player in DRAM.
  2. The attack, defense, and HP of a monster.

- **Use Skill**
  Input:
  1. Select the No. of player in DRAM.
  2. The list of the MP consumed by the skill.

- **Check Inactive**
  Input:
  1. Today's Date
  2. Select the No. of player in DRAM.

Information:

Table 1: The Content of Player's information (Store in the DRAM)

Bits	Player's information	Max. Value (Dec.)
16	Exp	65535
16	MP	65535
16	HP	65535
16	Attack	65535
16	Defense	65535
8	Login Date (Month)	12
8	Login Date (Day)	31

Note:

1. The order of content in the player in the DRAM from MSB to LSB is: **[MSB] {HP, Date (Month), Date (Day), Attack, Defense, Exp, MP} [LSB]**
2. February only has 28 days in this design.

Table 2: Scene Mode

Mode	Input	Exp_needed	Δ_{final_i}
Easy	2'b00	4095	$\Delta_{final_i} = \Delta_i - \left\lfloor \frac{\Delta_i}{4} \right\rfloor$
Normal	2'b01	16383	$\Delta_{final_i} = \Delta_i$
Hard	2'b10	32767	$\Delta_{final_i} = \Delta_i + \left\lfloor \frac{\Delta_i}{4} \right\rfloor$

Table 3: Training type

Training type	Input	Formula
A	0	$\Delta_i = \left\lfloor \frac{MP + HP + Attakc + Defense}{8} \right\rfloor$
B	1	$A0, A1, A2, A3 = ascended.sort(MP, HP, Attack, Defense)$ $\Delta_{A0} = A2 - A0, \Delta_{A1} = A3 - A1$
C	2	$if(Attribute_i < 16383), \Delta_i = 16383 - Attribute_i$
D	3	$\Delta_i = \min\left((3000 + ((65535 - Attribute_i) \gg 4)), 5047\right)$

Table 4: Damage in Battle

Damage	Formula
Damage_to_player	Monster.Attack – Player.Defense
Damage_to_monster	Player.Attack – Monster.Defense

Table 5: HP_temp in Battle

HP_temp	Formula
Player.HP_temp	If (Damage_to_player >0) Player.HP − Damage_to_player else Player.HP
Monster.HP_temp	If (Damage_to_monster >0) Monster.HP – Damage_to_monster else Monster.HP

Table 6: Attributes of the player after Battle

Result	Condition	Exp_new/MP_new/HP_new/Attack_new/Defense_new
Win	Player.HP_ temp > 0 & Monster.HP_temp ≤ 0	Exp_new = Exp + 2048 MP_new = MP + 2048 HP_new = HP_temp
Loss	Player.HP_ temp ≤ 0	Exp_new = Exp – 2048 HP_new = 0 Attack_new = Attack – 2048 Defense_new = Defense – 2048
Tie	Player.HP_ temp > 0 & Monster.HP_temp > 0	HP_new = HP_temp

Table 7: The corresponding value of actions and warning messages

ACTION		WARNING MESSAGE	
Login	3’b000	Saturation_Warn	3’b101
Level up	3’b001	Exp_Warn	3’b010
		Saturation _Warn	3’b101
Battle	3’b010	HP_Warn	3’b011
		Saturation _Warn	3’b101
Use skill	3’b011	MP_Warn	3’b100
Check Inactive	3’b100	Date_Warn	3’b001
All action		No_Warn	3’b000

Note: The number in parentheses indicates the priority. The smaller, the higher. If the operation results in several warnings at the same time, output the one with the highest priority.

The following are some rules about the actions:

1. The action will not be complete if the action outputs the warning message.
2. There will be 1~4 cycles for each input.
3. The date should adhere to the real calendar, that is, 03/21, 07/22, 11/13, 11/15, 12/24 is legal; 02/29, 04/31, 05/00 is illegal.
4. Keep the 'Complete' signal low until finishing an operation without any warnings.
5. The system will perform saturated arithmetic, meaning if the addition result exceeds 65535, it will be clamped at 65535, and if the subtraction result goes below 0, it will be clamped at 0.

Login

6. When **Sel_action_valid** is high and input signal **D** = 0 (Login), conduct this action.
7. The input order for **D** will be: Today’s Date → No. of player in DRAM.
8. Replace the player’s login date to INPUT’s given date.
9. Check the login date of the selected player. If the login is consecutive, increase the Exp by 512 and the MP by 1024.
10. If the updated Exp or MP exceeds the upper limit, set it to the maximum value (65535) and raise the warning message “Saturation_Warn (3’b101)”.
11. If no warnings are issued, pull the signal 'Complete' to HIGH, and set the warning message to “No_Warn” (3’b000).

Level Up

12. When **Sel_action_valid** is high and input signal **D** = 1 (Level Up), conduct this action.
13. The input order for **D** will be: Training type → Mode → No. of player in DRAM.
14. Check whether the player's Exp is greater than or equal to the mode's Exp_need. If not, end the level up and raise the warning message "Exp_Warn (3'b010)".
15. First calculate Δ_i according to the training type, then adjust Δ_i based on the mode to get the final upgrade result ($Attribute_{final_i} = Attribute_i + \Delta_{fianl_i}$).
16. If any of the final upgraded attributes exceed the upper limit, set it to the maximum value (65535) and raise the warning message "Saturation_Warn (3'b101)".
17. If no warnings are issued, pull the signal 'Complete' to HIGH, and set the warning message to "No_Warn" (3'b000).

Battle

18. When **Sel_action_valid** is high and input signal **D** = 2 (Battle), conduct this action.
19. The input order for **D** will be No. of player in DRAM → Monster's attack → Monster's Defense → Monster's HP.
20. If the player's HP is equal to 0, raising the warning message "HP_Warn (3'b011)" and end the Battle.
21. Determine the result of the Battle according to Tables 4, 5, and 6, and update the attributes based on the result.
22. If any of the attributes exceed the upper limit or fall below the lower limit, the warning message will be "Saturation_Warn (3'b101)".
23. If the updated attribute exceeds the upper limit, set it to the maximum capacity the attribute can hold (65535); if it falls below the lower limit, set it to the minimum capacity the attribute can hold (0).
24. If no warnings are issued, pull the signal 'Complete' to HIGH, and set the warning message to "No_Warn" (3'b000).

Use Skill

25. When **Sel_action_valid** is high and input signal **D** = 3 (Use Skill), conduct this action.
26. The input order for **D** will be No. of player in DRAM →The list of the MP consumed by the skills.
27. Using a skill consumes MP. First calculate the maximum number of skills that can be used with the player's available MP. If the player cannot use any skills, the warning message will be "MP_Warn (3'b100)".

28. Then update the player's MP based on the skills used (MP_new = MP – Σ MP_consumed).
29. If no warnings are issued, pull the signal 'Complete' to HIGH, and set the warning message to "No_Warn" (3'b000).

Check inactive

30. When Sel_action_valid is high and input signal **D** = 4 (Check Inactive), conduct this action.
31. The input order for D will be Today's Date → No. of player in DRAM.
32. Check the login date of the selected player in DRAM. If today's date is more than 90 days later than the last login date, stop the program and raise the warning message to "Date_Warn (3'b001)".
    For example:
    (a) Login date: 12/31, today: 1/1 → one day → ok
    (b) Login date: 1/1, today: 4/1 → 90 days → ok
    (c) Login date: 1/1, today: 4/2 → 91 days → Date_Warn
33. If no warnings are issued, pull the signal 'Complete' to HIGH, and set the warning message to "No_Warn" (3'b000).

System Integration Silicon Implementation Group. NCTU ELECTRONICS
Multimedia Wireless FEC Security & PLL

Design Block Diagram

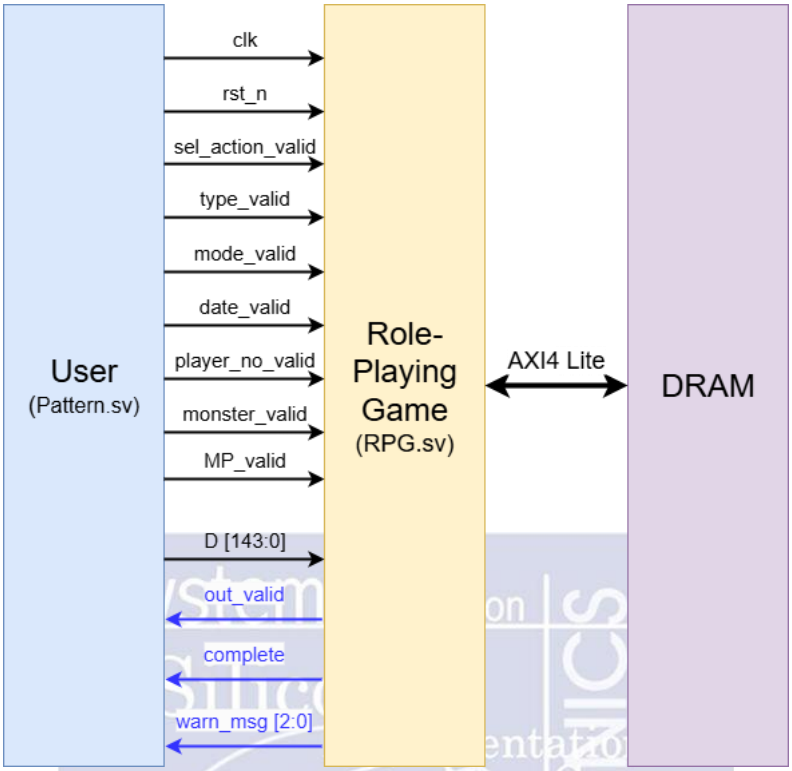

Figure 1: The Design Block Diagram

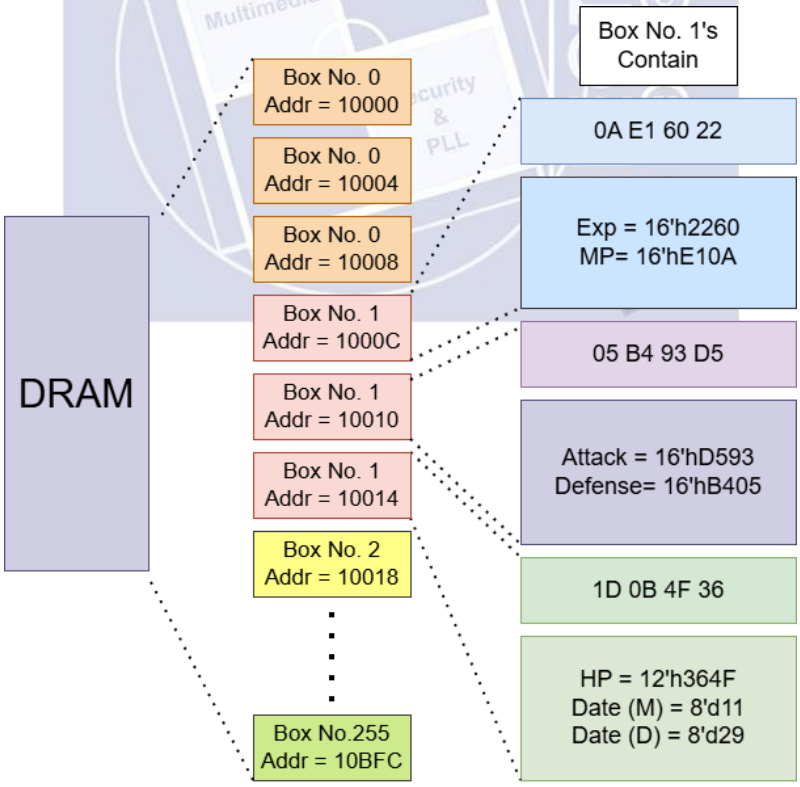

Figure 2: Dram data Content Diagram

Design Rules

1. After **rst_n**, all output signal in RPG.sv should be set to 0.
2. Please initialize DRAM at the beginning.
3. DRAM_R_latency, DRAM_W_latency, DRAM_B_latency in pseudo_dram.sv is now set to 1, but you can modify it. **TA will change the value (1<= value <=100) while demo.**
4. The pattern will be inserted using **7 valid signals + 1 data signals**.
5. You need to raise **out_valid** only when all input_valid signals are transmitted and your design has done current input operation.
6. We can get selected player from DRAM via AXI Lite protocol.
7. If action is '**Login**'
   - **Success**: When out_valid is HIGH, "complete" should be HIGH and "warning message" should be "No_Warn" (3'b000).
   - **Fail:**
     i. **Out of range:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Saturation_Warn (3'b101)".

8. If action is **Level Up**:
   - Update the attributes of the selected player.
   - **Success:** When out_valid is HIGH, "success" should be HIGH and "warning message" should be "No_Warn" (3'b000).
   - **Fail:**
     i. **Insufficient EXP**: When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Exp_Warn (3'b010)".
     ii. **Out of range:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Saturation_Warn (3'b101)".

9. If action is '**Battle**':
   - Update the attributes according to battle result.
   - **Success**: When out_valid is HIGH, "success" should be HIGH and "warning message" should be "No_Warn" (3'b000).
   - **Fail:**
     i. **Insufficient HP**: When out_valid is HIGH, "complete" should be LOW and "warning message" should be "HP_Warn (3'b011)"
     ii. **Out of range**: When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Saturation_Warn (3'b101)".

10. If action is '**Use Skill**':
    - Update the MP.
    - **Success**: When out_valid is HIGH, "success" should be HIGH and "warning message" should be "No_Warn" (3'b000).
    - **Fail**:
      i. **Insufficient MP:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "MP_Warn (3'b100)".

11. If action is '**Chec Inactive**':
    - **Success**: When out_valid is HIGH, "success" should be HIGH and "warning message" should be "No_Warn" (3'b000).
    - **Fail**:
        i. **Inactive for over 90 days:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Date_Warn (3'b001)".

12. The 7 valid input signals will not overlap with each other.
13. The next valid input signal will be valid for 1~4 cycles after the valid input signal or out_valid signal fall.
14. Out_valid cannot overlap with the 7 input valid signals (sel_action_valid, type_valid, mode_valid, date_valid, player_no_valid, monster_valid, MP_valid).
15. No checks will be performed by the TA for any overlaps between AXI4 Lite signals and the 7 input valid or out_valid signals.
16. Out_valid should be high for exactly one cycle.
17. Out_valid can only be high after giving all necessary input valid signals.
18. TA will check the output signal when the out_valid is high.
19. If actions complete, complete should be high and warn_msg should be 3'b000.
20. If action is not complete, complete should be the low, warn_msg should be corresponding value.
21. The next operation will be valid for **1-4** cycles after the out_valid fall.
22. The system will NOT check the data stored in DRAM.
23. To understand how AXI4LITE works, please refer to the Lab09_Exercise_Note.pdf

Input: RPG.sv

Signal	Width	from	Note
clk	1	testbench	System clock
rst_n	1	pattern	Asynchronous reset active low reset. Every output signal should be zero after **rst_n**.
sel_action_valid	1	pattern	High when input means select the action.
type_valid	1	pattern	High when input means type of training.
mode_valid	1	pattern	High when input means the mode.
date_valid	1	pattern	High when input means today's date.
player_no_valid	1	pattern	High when input means the number of player.
monster_valid	1	pattern	High when input means the attributes of the monster. (Will pull HIGH 3 times for monster's attack, monster's defense, and monster's HP
MP_valid	1	pattern	High when input means the list of the MP consumed by the skill (Will pull HIGH 4 times for 4 skills)

D[143:0]	144	pattern	Represents the contents of the current input. = {141'bX, Action} (inf.D.d_act [0]) = {142'bX, Training_Type} (inf.D.d_type[0]) = {142'bX, Mode} (inf.D.d_mode[0]) = {135'bX, Month[3:0], Day[4:0]} (inf.D. d_date[0]) = {136'bX, Player No.} (inf.D.d_player_no[0]) = {128'bX, Moster_attributes} (ins.D.d_attribute[0]) = {128'bX, Comsumed_MP}(inf.D. d_attribute[0]
AR_READY	1	DRAM	AXI Lite signal
R_VALID	1	DRAM	AXI Lite signal
R_DATA	96	DRAM	AXI Lite signal
R_RESP	2	DRAM	AXI Lite signal
AW_READY	1	DRAM	AXI Lite signal
W_READY	1	DRAM	AXI Lite signal
B_VALID	1	DRAM	AXI Lite signal
B_RESP	2	DRAM	AXI Lite signal

Output: RPG.sv

SiliconX

Signal	Width	Send to	Note
out_valid	1	pattern	Should set to high when your output is ready. **out_valid** will be high for **only one** cycle.
warn_msg	3	pattern	warn_msg will be 3'b000 (No warn) if operation is complete, else it needs to be corresponding value.
complete	1	pattern	1'b1: operation complete 1'b0: some warning occurred
AR_VALID	1	DRAM	AXI Lite signal
AR_ADDR	17	DRAM	AXI Lite signal
R_READY	1	DRAM	AXI Lite signal
AW_VALID	1	DRAM	AXI Lite signal
AW_ADDR	17	DRAM	AXI Lite signal
W_VALID	1	DRAM	AXI Lite signal
W_DATA	96	DRAM	AXI Lite signal
B_READY	1	DRAM	AXI Lite signal

Specifications

Top module

1. Top module name: RPG (file name: **RPG.sv**)
2. Your design should trigger at positive edge clock, and the pattern will trigger and receive input/output signal at negative edge clock.

Reset

3. It is asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals.
4. The reset signal(**rst_n)** would be given only once at the beginning of simulation. All output signals (including RPG.sv) should be reset after the reset signal is asserted.
5. All signal should be reset to 0.

Design Constraints

6. The maximum clock period is **15 ns**.
7. Your latency should be **less than 1000 cycles** for each operation.
8. All outputs (including RPG.sv) are synchronized at clock rising edge.
9. The type defined in Usertype.sv by TA **should not be modified, but you are encouraged to define a new datatype if needed**.
10. Out_valid can only be high for exactly one cycle.
11. You CANNOT use any designware IP and SRAM in this lab.

Synthesis

12. The input delay and the output delay are **'0.5*clock period**'.
13. The output load should be set to **0.05.**
14. The synthesis result of data type cannot include any **LATCH.**
15. Synthesis time should less than 30 minutes.
16. The area should be **less than 200000.**

Gate level simulation

17. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

Supplement

18. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
19. Don't write Chinese or other language comments in the file you sent.
20. Verilog command for **design compiler optimizations are forbidden**.
21. **Any error messages** during synthesize and simulation, regardless of the result will lead to failure in this lab.
22. Any form of display or printing information in Verilog design is forbidden. You may use this methodology during debugging, but the file you turn in **should not contain any coding that is not synthesized.**

Grading

- **70%: Function**
- **30%: Performance: Area x latency x cycle_time**

Note

- Submit your files in Lab09/EXERCISE/09_SUBMIT
  a) 1st_demo deadline: 2025/11/24 (Mon.) 12:00:00
  b) 2nd_demo deadline: 2025/11/26 (Wed.) 12:00:00
- **Please upload the following files under 09_SUBMIT:**
  - **In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.**

    ```
    ./01_submit 8.0
    ```
  - RPG.sv, Usertype.sv
  - If your files violate the naming rule, you will get 10 deduction points.

※ **Since Lab10 is about pattern with SystemVerilog, if you need to share your pattern with others, use the following command to encrypt your PATTERN. It will generate "PATTERN.svp" file.**

```
[~/Lab09/Exercise/00_TESTBED]$ ./00_ENC_PATTERN
```

Since we have provided the way to protect your file, there will be no excuse for plagiarism.

■ The detail about AXI4 Lite protocol, Reference Waveform and Note please refer to appendix.

■ You need to generate dram.dat yourself and the DRAM address range starts from @10000 and ends at @10BFF.

```
@10000
20 67 D7 E6
@10004
B4 11 A9 EC
@10008
17 06 75 4B
```

to

```
@10BF4
49 D7 8B 90
@10BF8
28 81 67 21
@10BFC
15 07 DE F4
```